

App Development Project Report

App Dev Project Report

1. Student Details

Name: Yuvraj Kashyap

Roll Number: 25F2004244

Email: 25f2004244@ds.study.iitm.ac.in

About Me: I am a Diploma Level student at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

To design and build a web-based **Trekking Management Application** that enables users to **explore, join, and manage trekking events**, while also allowing administrators and staff to organize treks, manage participants, and ensure smooth coordination within a growing trekking community.

Approach:

The application was developed using **Flask** as the backend framework with a structured and modular design. It allows users to browse available treks, register and participate in them, and interact within a community of trekkers.

Administrative functionalities include managing trek details, handling user registrations, approving staff members, and managing platform activity. The system integrates both backend logic and frontend templates to provide a seamless and user-friendly experience.

3. AI/LLM Declaration

AI tools were primarily used for identifying bugs, understanding errors, and assisting in debugging the code. Minor help was taken for UI improvements and small code snippets. The overall project architecture, database design, and core application logic were developed independently.

The extent of AI/LLM usage is estimated to be around 15–20%, mainly limited to debugging assistance, minor code suggestions, and UI refinements. All final implementation, integration, and decision-making were carried out manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework used for routing and application logic
SQLAlchemy	ORM used for database modeling and handling data operations
Jinja2	Template engine for rendering dynamic HTML pages
SQLite	Lightweight database used to store users, treks, and booking data
HTML/CSS	Used to design and structure the frontend of the application

5. Database Schema / ER Diagram

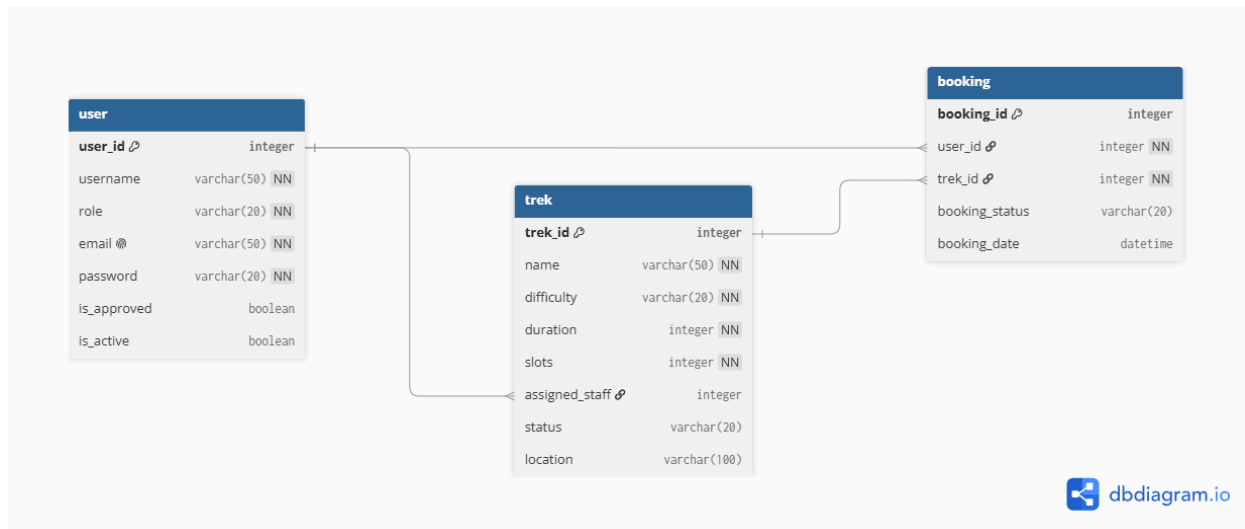
Tables:

1. **User** — stores account details for all roles (user_id, username, email, password, role, is_approved, is_active)
2. **Trek** — stores trek details (trek_id, name, difficulty, duration, slots, status, location, assigned_staff)
3. **Booking** — logs trek booking requests (booking_id, user_id, trek_id, booking_status, booking_date)

Relationships:

- One-to-Many → **User** → **Booking** (a user can make many bookings)

- One-to-Many → Trek → Booking (a trek can have many bookings)
- One-to-Many → User (as Staff) → Trek (a staff member can be assigned to many treks; nullable — a trek may have no staff assigned)



6. Route / URL Endpoints

This project does not use REST APIs. All functionalities are implemented using server-side rendering with Flask routes and Jinja2 templates.

Route-level access control is enforced via Flask session checks (`session['role']`) at the start of each protected route, rather than a token-based authentication scheme, consistent with the server-rendered session-based architecture of this application.

Route	Method	Access	Description
/	GET	Public	Renders the home/landing page
/user_login	GET, POST	Public	Displays login form; authenticates a user and starts a session
/staff_login	GET, POST	Public	Displays login form; authenticates staff (blocks unapproved staff)
/admin_login	GET, POST	Public	Displays login form; authenticates the admin account

<code>/signup</code>	GET, POST	Public	Registers a new user or staff account (staff requires admin approval)
<code>/admin_dashboard</code>	GET, POST	Admin only	Main admin panel — dashboard stats, and manage treks/staff/users/bookings sections (supports search via query params)
<code>/add_trek</code>	POST	Admin only	Creates a new trek
<code>/approve_staff /<id></code>	GET	Admin only	Approves a pending staff signup request
<code>/remove_staff /<id></code>	GET	Admin only	Revokes a staff member's approval
<code>/blacklist /<id></code>	GET	Admin only	Deactivates (blacklists) a staff or user account
<code>/remove_staff_blacklist /<id></code>	GET	Admin only	Reactivates a blacklisted staff account
<code>/remove_user_blacklist /<id></code>	GET	Admin only	Reactivates a blacklisted user account
<code>/delete_trek /<id></code>	POST	Admin only	Permanently deletes a trek and its bookings
<code>/edit_trek /<trek_id></code>	GET, POST	Admin only	Displays and processes edits to a trek's details, status, and staff assignment
<code>/user_dashboard</code>	GET, POST	User only	Main user panel — booking history, browse/search/book treks, profile settings
<code>/book_trek /<id></code>	POST	User only	Creates a pending booking request for a trek
<code>/staff_dashboard</code>	GET, POST	Staff only	Main staff panel — assigned treks, registered users per trek, profile settings
<code>/update_slots_staff /<id></code>	POST	Staff only	Updates total slot count for a trek assigned to that staff member

<code>/update_status</code> <code>/<id></code>	POST	Staff, Admin	Updates a trek's status (Open/Ongoing/Closed/Completed)
<code>/update_booking_status</code> <code>/<id></code>	POST	Staff, Admin	Approves or declines a pending booking request
<code>/logout</code>	GET	Any logged-in role	Clears the session and redirects to home

7. Architecture and Features

Architecture Overview:

- `main.py` — Flask app entry point; all routes (auth, admin, staff, user)
- `model.py` — SQLAlchemy models: `user`, `trek`, `booking`
- `/templates` — Jinja2 HTML templates (login, signup, home, role dashboards), extending `base.html`
- `/static` — `main.css` for styling
- `database.db` — SQLite, auto-created via `db.create_all()`

Implemented Features:

- Role-based registration/login (user, staff, admin) with session-based access control
 - Staff signup requires admin approval
 - Admin: create/edit/delete treks, assign staff, set trek status
 - User: browse/search treks, view live slot availability, book treks
 - Booking approval workflow with slot-limit enforcement
 - Staff dashboard: assigned treks, registered users per trek, slot/status management
 - Admin: search & block/unblock staff and users
 - Search across treks, staff, and users
 - Profile settings (username/email/password) for staff and users
-

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1etK78LcF6N310iZ-jBbMi6wXsj5hL3PG/view?usp=sharing>